

# GrowthPilot — Agentic Commerce Intelligence

Complete Technical Architecture, File Directory, Data Flow & System Specification

## 1. Executive Summary & Technology Stack

**GrowthPilot** is an autonomous e-commerce intelligence and growth acceleration platform. It connects to payment gateways (Razorpay Test Mode), normalizes transactional telemetries into a PostgreSQL relational database, computes multi-factor predictive scores (RFM Intent, Churn Risk, Hybrid Recommendations), and exposes an autonomous AI agent with human-in-the-loop governance.

| Layer      | Technologies                                                              | Key Role                                                      |
|------------|---------------------------------------------------------------------------|---------------------------------------------------------------|
| Frontend   | React 19, TypeScript 5.8, Vite 6, Tailwind CSS 4, TanStack Query v5       | Interactive single-page dashboard & AI agent workspace        |
| Backend    | Java 22, Spring Boot 3.4.1, Spring Data JPA, Hibernate, SpringDoc OpenAPI | REST API, authentication, predictive scoring engines          |
| Security   | Spring Security 6, Stateless HMAC-SHA256 JWT, BCrypt password hashing     | Multi-tenant workspace isolation & endpoint protection        |
| Database   | PostgreSQL 16 Alpine, Docker Compose volume persistence                   | Relational data tier storing tenants, orders, payments, carts |
| Payments   | Razorpay Java SDK (v1.4.6), Basic Auth over HTTPS                         | Ingests payments, orders, refunds, and customer checkouts     |
| AI / Agent | Gemini 1.5 Flash Provider + Deterministic Rule Engine + Tool Registry     | Autonomous growth interventions with approval workflow        |

## 2. Complete Project Directory Structure

The exact directory hierarchy of the GrowthPilot platform:

```
growthpilot/
|-- docker-compose.yml           # Multi-container orchestration (Postgres, pgAdmin, Backend, Frontend)
|-- README.md                   # System architecture overview & documentation
|-- backend/
|   |-- pom.xml                 # Maven dependencies: Spring Boot 3.4.1, Razorpay Java 1.4.6, JWT
|   |-- src/main/resources/
|   |   |-- application.yml      # JWT configuration, DataSource URL, Razorpay defaults
|   |   |-- schema.sql          # Core PostgreSQL DDL schema & relational constraints
|   |-- src/main/java/com/growthpilot/
|   |   |-- GrowthPilotApplication.java # Spring Boot entry point
|   |   |-- agent/tool/         # Concrete AI Agent Tools (GetAbandonedCartsTool, CreateOfferTool...)
|   |-- ai/GeminiAiProvider.java # Google Gemini LLM API integration layer
|   |-- controller/             # REST Controllers (Dashboard, Customers, Products, Agent, Carts...)
|   |-- dto/                   # Data Transfer Objects (AuthResponse, DashboardSummaryDto...)
|   |-- entity/                 # Relational JPA Entities (Business, Order, Payment, Customer, Cart...)
|   |-- razorpay/               # Razorpay Integration Connector Module
|   |-- repository/             # Spring Data JPA Repositories (Tenant-isolated queries)
|   |-- security/               # JWT Authentication filter, UserDetailsService & SecurityConfig
|   |-- service/                # Business logic (CustomerService, DashboardService, ChurnRiskService...)
|-- frontend/
|   |-- package.json            # React 19, Vite, TanStack Query, Lucide icons, Recharts
|   |-- vite.config.ts          # Vite build configuration & localhost:8080 API proxy
|   |-- src/
|   |   |-- main.tsx / App.tsx  # Application routing & TanStack QueryClientProvider
```

```
    |-- components/                # Reusable components (MetricCard, StatusBadge, Sidebar, Layout)
    |-- pages/                     # 11 Major screen modules (Dashboard, Customers, Carts, Copilot.
..)
    `-- services/api.ts            # Unified frontend API client with Bearer token interceptor
```

### 3. Razorpay Test Mode Ingestion Architecture

The synchronization engine (**RazorpaySyncService**) connects to the Razorpay Test API using tenant credentials and normalizes transactional records into PostgreSQL without duplicate entries:

```
[Razorpay Test API] --> (GET /v1/customers, /orders, /payments, /refunds)
    |
    v
[RazorpaySyncService.syncAll(businessId)]
    |
    |-- 1. Customers: Maps contact, email, and razorpay_customer_id to 'customers' table
    |-- 2. Orders: Matches order receipt ID and status to update existing store 'orders'
    |-- 3. Payments: Converts paise (amount/100.0), maps method (netbanking/card/upi), status & error_
reason
    `-- 4. Refunds: Links refund records directly to parent payments in 'refunds' table
    |
    v
[PostgreSQL Database] --> (DashboardService & AI Copilot read real synchronized metrics)
```

### 4. Feature to Code Implementation Map

| Feature Area            | Frontend Page                               | Backend Controller & Service                                      | PostgreSQL Tables                           |
|-------------------------|---------------------------------------------|-------------------------------------------------------------------|---------------------------------------------|
| Executive Dashboard     | DashboardPage.tsx                           | DashboardController.java<br>DashboardService.java                 | orders, payments, refunds, customers, carts |
| Customer Intelligence   | CustomersPage.tsx<br>CustomerDetailPage.tsx | CustomerController.java<br>CustomerService.java                   | customers, orders, customer_events          |
| Product Intelligence    | ProductsPage.tsx                            | ProductController.java<br>ProductService.java                     | products, order_items                       |
| AI Recommendations      | RecommendationsPage.tsx                     | RecommendationController.java<br>RecommendationService.java       | recommendations, products, customers        |
| Growth Opportunities    | OpportunitiesPage.tsx                       | GrowthOpportunityController.java<br>GrowthOpportunityService.java | growth_opportunities, carts, customers      |
| Cart Recovery Hub       | AbandonedCartsPage.tsx                      | CartController.java<br>CartService.java                           | carts, cart_items, customers                |
| Campaigns & Offers      | CampaignsPage.tsx                           | CampaignController.java                                           | campaigns                                   |
| AI Growth Copilot       | CopilotPage.tsx                             | AgentController.java<br>AiCopilotService.java                     | agent_actions, customers, products, carts   |
| Agent Approvals & Audit | AgentActivityPage.tsx                       | AgentController.java<br>AgentActionService.java                   | agent_actions                               |
| Razorpay Integration    | RazorpayPage.tsx                            | RazorpayIntegrationController.java<br>RazorpaySyncService.java    | payments, orders, customers, refunds        |
| Workspace Settings      | SettingsPage.tsx                            | WorkspaceController.java<br>DataSeedService.java                  | businesses, users                           |

### 5. AI Agent Execution & Governance Lifecycle

GrowthPilot enforces a strict **Human-in-the-Loop** safety boundary. Autonomous AI actions never execute destructive or customer-facing changes (such as discount code creation or email dispatches) without human approval:

- 1. User Prompts Copilot: "Find high-value abandoned carts and recover them"
- 2. AiCopilotService: Queries PostgreSQL context via GetAbandonedCartsTool
- 3. Agent: Formulates reasoning and generates an action proposal
- 4. AgentActionService: Persists action with status 'PENDING\_APPROVAL' in 'agent\_actions'
- 5. Growth Manager: Reviews proposal, parameters, and AI reasoning on /agent/activity screen
- 6. Decision Gate:
  - Click [Approve & Execute] -> Tool runs mutation, action status set to 'EXECUTED'
  - Click [Reject Action] -> Status set to 'REJECTED'
- 7. Immutable Audit Trail: Record remains permanent in PostgreSQL for compliance.

## 6. Data Origin & Classification Matrix

| Data Entity                   | Classification | Underlying Source of Truth                                                                 |
|-------------------------------|----------------|--------------------------------------------------------------------------------------------|
| Razorpay Payments             | REAL           | Live sync from Razorpay Test Mode API into PostgreSQL 'payments' table.                    |
| Total Store Revenue           | REAL / DERIVED | SQL aggregation: Captured payments + completed orders minus refunds.                       |
| Payment Failure Reasons       | REAL           | Directly extracted from Razorpay payment entity 'error_reason'.                            |
| Customer Intent Score (0-100) | DERIVED        | Calculated by PurchaseIntentService based on order frequency, recency, and spend velocity. |
| Customer Churn Risk           | DERIVED        | Evaluated dynamically (LOW/MED/HIGH) by ChurnRiskService from inactivity windows.          |
| AI Product Recommendations    | DERIVED        | Hybrid recommendation scorer: category affinity (0.40) + sales velocity (0.25).            |
| Abandoned Cart Win-Back Rate  | DERIVED        | Computed deterministically by CartService using cart value and buyer history.              |
| Product Catalog (Demo)        | MOCK / SEED    | Seeded into PostgreSQL to test intelligence engines when catalog integrations are absent.  |

## 7. Developer Mental Model: What Happens Step-by-Step

### Q: What happens when I click 'Sync Now' on the Razorpay page?

The browser sends an authenticated POST to /api/integrations/razorpay/sync. Spring Boot loads your saved Key ID and Secret from the database, initializes the Razorpay Java SDK, and calls fetchAll() on customers, orders, payments, and refunds. Each record is converted from paise into rupees, dates are safely parsed, and records are saved to PostgreSQL with duplicate checks.

### Q: What happens when the Executive Dashboard displays revenue?

The dashboard does not call Razorpay directly. Instead, DashboardService.java queries PostgreSQL for all captured payments and orders belonging to your workspace, subtracts any processed refunds, and sends the calculated sum (e.g. ₹500 or ₹2,000) to the React frontend, where MetricCard.tsx renders the formatted rupee amount.

### Q: What happens when I ask the AI Growth Copilot a question?

The AI does not hallucinate numbers. When you ask about failed payments or abandoned carts, AiCopilotService.java first triggers an internal tool (like GetFailedPaymentsTool), queries your actual PostgreSQL tables, and injects those exact numbers into the prompt. The response is strictly grounded in your store's real database records.